PipeDoctor
Git and CI

Sprint Step 3: put the project under version control,
and let GitHub run the tests on every push.

Sprint 1 · Step 3 · 8 October 2026
Files added: `.github/workflows/ci.yml`, `.gitattributes`, decision D12.

1 What we do, and why

Goal. A GitHub repository for PipeDoctor, outside OneDrive, where every push automatically runs the 8 unit tests in a clean machine.

Reason	What it gives us
History and undo	Every change is a commit you can read, compare and revert. The journal says *what* we did; git says exactly *which lines*.
Pull requests	PipeDoctor's whole output is a pull request that a human approves. The agent needs a repository to open it on (tool `open_pr`, sprint Step 17).
Automatic safety net	CI runs the tests on a clean Linux machine, not on yours. "Works on my PC" stops being an excuse.
Foundation for the gate	In sprint Step 26, the same mechanism will run the benchmark and *block* a regression. We start with the habit now.
Leave OneDrive	`data/` is now 1.3 GB and Delta files change constantly. OneDrive would sync them and can lock files in use.

2 The picture

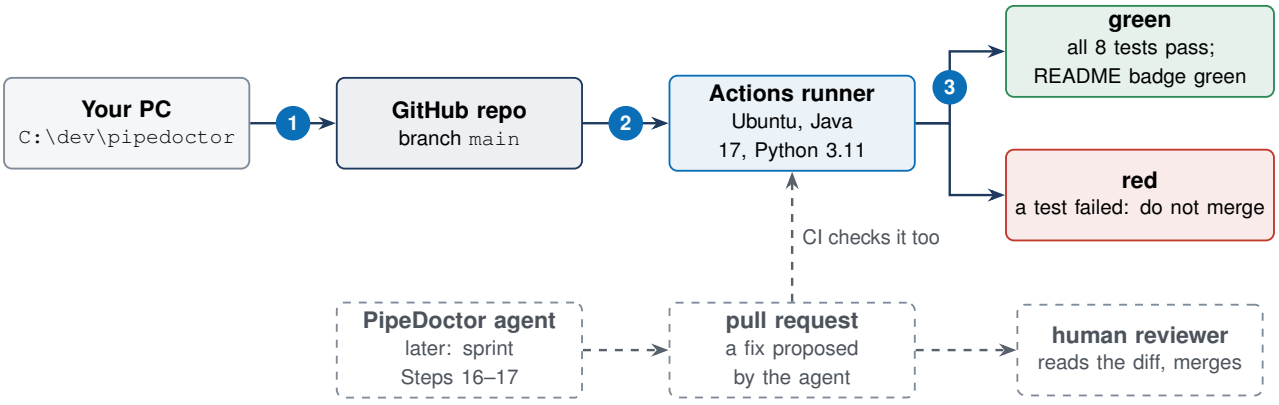


Figure 1. 1 `git push` 2 the push triggers `ci.yml` 3 the runner reports a result. Dashed grey = what comes later: the agent's pull requests go through the same CI.

3 The files I added

`.github/workflows/ci.yml` — the CI recipe

Lines	What they say
3--6	Run on every push to `main` and on every pull request. The second part matters later: the agent's PRs get tested before a human looks at them.
8--9	`permissions: contents: read`. The job can read the code and nothing else (least privilege, the same idea as PipeDoctor's guardrails).
11--13	If you push twice quickly, cancel the older run on the same branch.
16--19	One job on a fresh `ubuntu-latest` machine, stopped after 15 minutes if stuck.
21	`actions/checkout`: download your code onto the runner.
24--27	Install Java 17 (Temurin). Spark needs the JVM, exactly like `docker/Dockerfile` line 5.
29--33	Install Python 3.11; pip downloads are cached, keyed on `docker/requirements.txt`.
35--36	`pip install -r docker/requirements.txt`: the *same pinned versions* as the Docker image (PySpark 3.5.3, Delta 3.2.1, pytest 8.3.3).
39--42	Run `python -m pytest pipelines/tests -q`, the same test command as on your PC.

Why the runner does not build the Docker image. It would add about 2.5 minutes for no gain: the tests use plain Spark without Delta (`pipelines/tests/conftest.py:8`), and the dependency versions are pinned to the same numbers. The cost is a small risk of drift between the two environments; we accept it and will revisit it in sprint Step 26.

`.gitattributes` and `.gitignore`

- `.gitattributes`: `* text=auto eol=lf` keeps Unix line endings. Windows would otherwise write CRLF, which breaks shell scripts inside Docker. You may see the warning `LF will be replaced by CRLF` on Windows; it is harmless.
- `.gitignore` (already there): excludes `data/`, `spark-warehouse/`, `.env`, Terraform state. The 1.3 GB of data never goes to GitHub; anyone can recreate it with `scripts/download_taxi_data.py`.

Decision D12 (`docs/decision-log.md`)

Repository `Aymannel04/pipedoctor`, **private** until the release (Gate 4), because `docs/` holds interview material and drafts. GitHub Actions allows enough free minutes on private repositories for unit tests. At the release we review `docs/` and make it public: a public repository is the real portfolio piece.

4 What you run, in order

Open **PowerShell**. Lines starting with `#` are comments.

A. Copy the project out of OneDrive

```
# close VS Code and any running Docker command first
mkdir C:\dev
robocopy "C:\Users\Ayman\OneDrive\Desktop\Pipe Doctor\pipedoctor" C:\dev\pipedoctor /E \
    /XD spark-warehouse .pytest_cache __pycache__ /XF *.pyc
cd C:\dev\pipedoctor
code .
```

`robocopy` reports success with exit codes 0 to 7; only 8 or more means an error. The copy includes `data/`, so you do not need to download or rebuild anything. Keep the OneDrive copy for now as a backup.

B. Create the local repository

```
git init -b main
git config user.name "Ayman"
git config user.email "YOUR-GITHUB-EMAIL"
git add .
git status
```

Check the `git status` output before committing: it must list `pipelines/`, `docs/`, `docker/`, `.github/`, and must *not* list `data/`. If `data/` appears, stop and show me.

```
git commit -m "Steps 1-3: project skeleton, local lakehouse, CI"
```

C. Create the empty repository on GitHub

In the browser: `https://github.com/new`. Owner `Aymannel04`, name `pipedoctor`, **Private**, and leave *all* the "initialise with" boxes unticked (no README, no `.gitignore`, no licence), otherwise the first push conflicts.

D. Push

```
git remote add origin https://github.com/Aymannel04/pipedoctor.git
git push -u origin main
```

Git for Windows opens a browser window to log in to GitHub the first time; accept it.

E. Watch the CI

Open `https://github.com/Aymannel04/pipedoctor/actions`. The workflow `CI` starts by itself. Expect 2 to 4 minutes (the first run downloads 317 MB of PySpark). A green tick means the tests pass on a clean machine.

5 Checklist and what to send me

- ☐ `git status` did not list `data/`.
- ☐ `git push` finished without error.
- ☐ The Actions tab shows a green `CI` run. **Send me a screenshot or the status (green or red).**
- ☐ Open the repository page: the README shows the architecture picture.

6 If something goes wrong

Symptom	Cause and fix
`fatal: detected dubious ownership`	Git distrusts the folder owner. Run the `git config --global --add safe.directory` command that the message prints.
`remote: Repository not found` on push	The GitHub repository was not created yet, or the name differs. Check step C.
`rejected ... fetch first`	You ticked "add a README" on GitHub. Delete the repository, recreate it empty.
Push is very slow or huge	`data/` got added. Run `git rm -r --cached data` then amend the commit.
CI is red on the *first* run	Read the log in the Actions tab. This is real practice: it is the same kind of log PipeDoctor will read. Paste the last 30 lines to me.
`LF will be replaced by CRLF`	Harmless warning, see `.gitattributes`.

7 After this step

Once CI is green, delete the `data/` folder inside the OneDrive copy to stop the sync. From then on we work in `C:\dev\pipedoctor`; I will ask for access to that folder so I can keep writing files there. Next is **sprint Step 4**: an Airflow DAG that runs the pipeline and writes a failure event when it breaks.